
ForL0 State Backend

设计说明书

v3.0

2026 年 7 月

目 录

1 整体架构设计	1
1.1 总体架构	1
1.2 状态访问优化路径	1
1.3 原生状态表组织	2
1.4 组件解耦与集成	2
2 架构设计细节	3
2.1 可复用组件与扩展组件	4
2.1.1 ForL0StateBackend 与 ForL0KeyedStateBackend	4
2.1.2 PriorityQueueManager / HeapPriorityQueue	4
2.1.3 NativeEngine	4
2.2 关键设计组件	5
2.2.1 StateEngine	5
2.2.2 状态布局与命名空间组织	5
2.2.3 SwissTable	5
2.2.4 L0 Hot-Key Cache	6
2.2.5 状态访问策略与 JNI 快路径	8
2.2.6 透明融合边界与状态内部快路径	8
2.2.7 v3.0 覆盖边界与离线复跑反馈	9
2.2.8 类型分析与布局描述	9
2.3 原生状态组织实现	10
2.3.1 结构组成	10
2.3.2 RowData 快路径实现	10
2.3.3 主要操作算法	10
2.4 ForL0State 实现	14
2.4.1 ForL0ValueState	14
2.4.2 ForL0ListState	14
2.4.3 ForL0MapState	15
2.4.4 ForL0ReducingState 与 ForL0AggregatingState	15
3 容错机制设计	15
3.1 快照发起	15

3.2	快照采集.....	16
3.3	Copy-On-Write 一致性模型.....	16
3.4	Savepoint 与状态恢复	16
3.5	资源回收与异常处理	17
4	插件式集成设计	17
4.1	加载机制.....	17
4.2	运行时接入边界	17
5	总结	18

1 整体架构设计

ForL0 State Backend 面向 Apache Flink 的 Keyed State 场景设计，目标是在保持 Flink 状态接口、命名空间语义和 Checkpoint 机制兼容的前提下，将状态访问的主要数据路径收敛到一套原生状态引擎中，通过类型特化、缓存友好的哈希

系统采用“Java 控制层 + JNI 桥接层 + C++ 原生状态引擎”的三层架构。Java 层承担接口适配、状态注册和用户函数执行，native 层承担状态组织、命名空间隔离、哈希存储以及快照恢复

1.1 总体架构

系统围绕 StateEngine -> StateTable -> KeyGroup -> Namespace -> SwissTable 的层级展开。一个任务实例对应一个 native StateEngine；每个状态描述符在 native 侧注册为一个 typed StateTable<K,V>；每个 StateTable 再按 key-group 和 namespace 组织具体状态条目；最底层的键值映射由 SwissTable<K,V> 完成。

整体职责可概括如下：

- Java 控制层负责与 Flink 运行时对接，管理状态对象生命周期、serializer 分析、状态注册和快照调度。
- JNI 桥接层负责共享库装载、handle 透传以及按类型划分的 native 调用入口。
- C++ 原生数据层负责状态条目的权威存储、哈希组织、命名空间管理、Copy-On-Write 快照和恢复写入。

这一结构的关键意义在于，状态主数据面已经完全位于 native 侧，Java 层不再维护一套独立的状态表副本，也不再承担状态快照的主序列化职责。

1.2 状态访问优化路径

系统专门设计了一层状态访问优化路径，用于缩短热点场景上的分支判断、对象封装和序列化链路。该优化路

- Java 状态对象在初始化阶段预计算访问策略，将 key、namespace 与 value 的组合固化为固定分支。
- NativeEngine 按高频类型组合暴露窄接口，例如 long key、int key、TimeWindow namespace、typed inner map 等专用路径。
- RowData key 通过 RowDataKeyAccessor 识别为单字段 primitive、固定长度多字段或 generic bytes 三类编码方式。
- 对部分 BinaryRowData 或 bytes value，native 层支持零拷贝读取，减少中间 byte[] 和对象重建成本。
- 在 JNI 热路径的最外层，系统默认启用基于鲲鹏 L0 Cache 用户态分区的 L0 Hot-Key Cache，对高频热键提供 4-10 cycle 级别的读写穿透缓存；缓存 miss

以及不符合类型条件的访问直接回落到 **SwissTable**，不改变数据一致性。

因此，系统的性能优化重点并不是改变 Flink 状态语义，而是在保持语义不变的前提下，尽量让热点访问 JNI 路由、最窄类型表示、最少数据复制，并使能够落在 L0 上的热键数据不再为每次访问支付 DRAM 延迟。

1.3 原生状态表组织

当前实现中，负责权威状态存储的核心结构是 native 侧的 **StateTable<K,V>** 与其内部的 **SwissTable<K,V>**。Java 层通过状态名、serializer 和类型布局将状态注册到 native 层，后续所有读写、清理、遍历和恢复都通过 state handle 回到对应的 **StateTable**。

状态表组织遵循以下规则：

- 每个 Flink 状态描述符在 **StateEngine** 中对应唯一的 state handle。
- 每个 state handle 对应一张 typed **StateTable<K,V>**。
- 每张 **StateTable** 按 key-group 划分内部状态分区，与 Flink key-group 路由模型一致。
- VoidNamespace 场景下直接访问 key-group 对应的 **SwissTable**；一般 namespace 场景下先路由到 namespace，再进入该 namespace 独立持有的 **SwissTable**。

这种组织方式同时满足了状态隔离、快照遍历稳定性和恢复写入粒度的要求，是整个后端一致性与可恢复

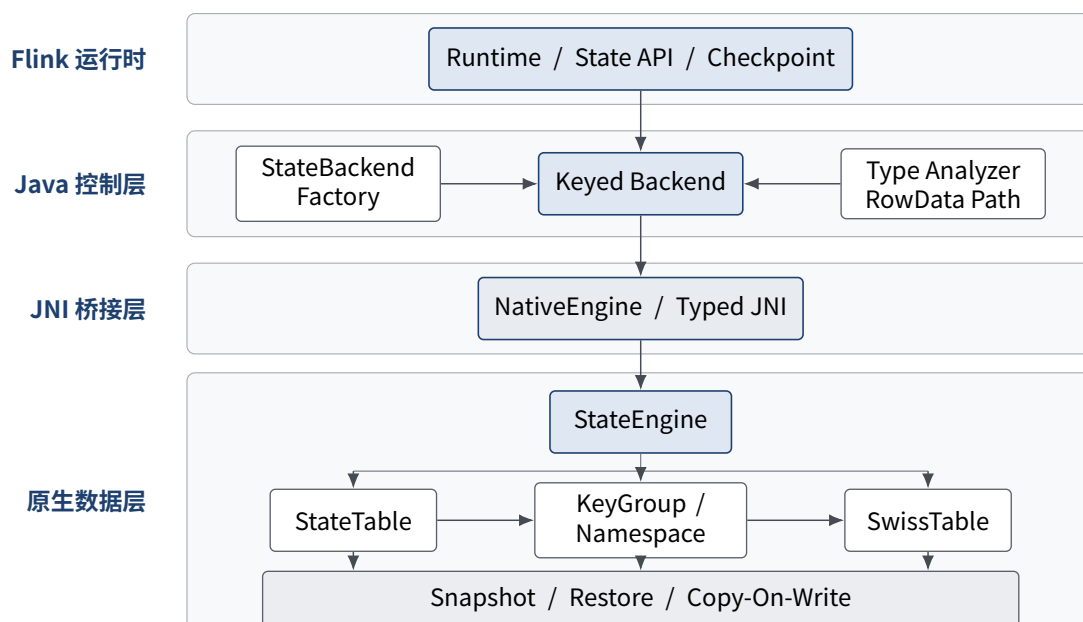


图 1 ForL0 State Backend 整体架构与层级关系

1.4 组件解耦与集成

ForL0 State Backend 的组件划分遵循“控制面与数据面分离”的原则。Flink 侧接口、Java 状态适配、JNI 桥接和 native 数据结构各自有明确职责边界，从而保证底层存储结构可以独立优化，而不破坏

核心组件的职责关系如下：

- `ForL0StateBackend` 与 `ForL0StateBackendFactory`：负责 Flink 侧后端创建与插件注册。
- `ForL0KeyedStateBackendBuilder`：负责 native 引擎创建、状态恢复初始化和 backend 实例构建。
- `ForL0KeyedStateBackend`：负责管理 engine handle、state handle、状态对象缓存以及快照协调。
- `NativeEngine`：负责定义 JNI 接口并承担共享库加载。
- `TypeAnalyzer` 与 `RowDataKeyAccessor`：负责类型识别、布局描述生成和 RowData 快路径支持。
- `StateEngine`、`StateTable` 与 `SwissTable`：共同构成 native 数据面，承担状态组织、访问、快照和恢复。
- `HotCacheManager` 与 `HotKeyCache`：在 L0 Cache 分区上构造热点键值对的 set-associative 读写穿透缓存，默认随后端启用；硬件或共享库不可用时自动降级为关闭，不做 heap 回补。

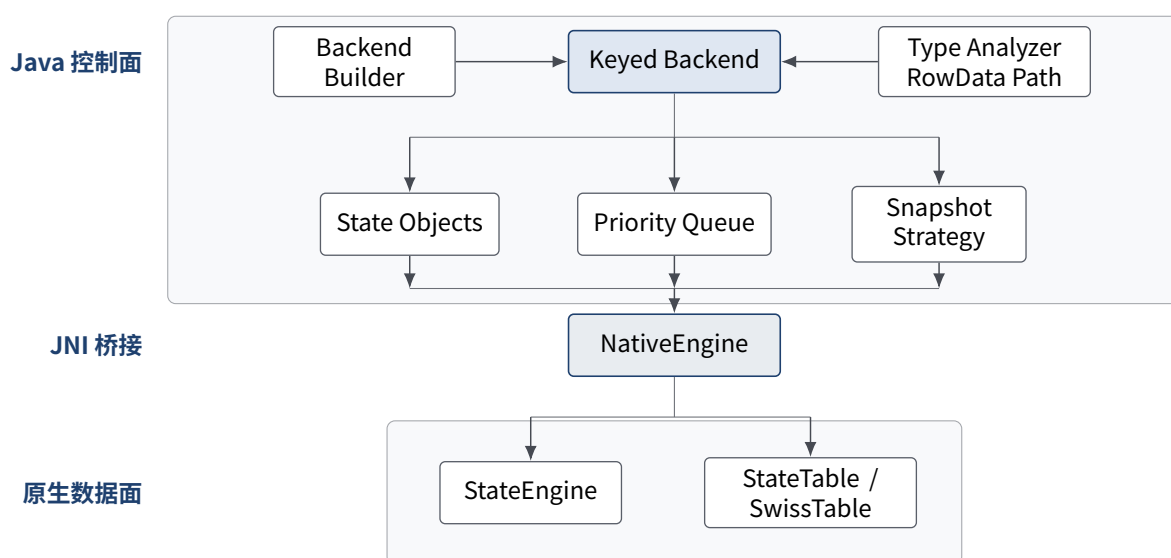


图 2 ForL0 State Backend 主要组件及其协作关系

2 架构设计细节

ForL0 State Backend 遵循 Flink `AbstractStateBackend` 与 `AbstractKeyedStateBackend` 的扩展方式。Java 层围绕状态描述符、serializer 和运行时上下文选择访问路径；native 层围绕 `StateTable<K,V>` 与 `SwissTable<K,V>` 组织读写、快照导出和恢复重建。

2.1 可复用组件与扩展组件

当前实现并不是完全绕开 Flink 原有状态框架，而是在与状态后端集成、优先队列管理、快照元信息和状态接入 Flink 的成熟机制，并将 Keyed State 的主数据路径替换为 ForL0 自身的 native 实现。

2.1.1 ForL0StateBackend 与 ForL0KeyedStateBackend

ForL0StateBackend 是 Flink 侧的后端入口，负责创建具体的 ForL0KeyedStateBackendBuilder。ForL0KeyedStateBackend 则是 Java 壳层中的核心控制组件，承担以下职责：

- 持有 native engine handle 和各状态的 native state handle。
- 缓存已创建状态对象与 state meta info。
- 在状态首次使用时完成 serializer 兼容性校验与 native 状态注册。
- 创建五类 Flink 状态实现，并把状态操作映射为相应 JNI 调用。

它在结构上是 Java 控制层与 native 数据层的直接衔接点，在语义上是 Flink 原生 Keyed-StateBackend 的兼容实现。

2.1.2 PriorityQueueManager / HeapPriorityQueue

定时器与优先队列部分继续沿用 Flink 原生实现。ForL0 State Backend 不改变优先队列内部行为与定时器语义，只替换主状态的键值存储路径。因此，Priority Queue 状态仍按照 Flink 既有方式参与快照和恢复。

2.1.3 NativeEngine

NativeEngine 是 Java 层访问原生状态引擎的统一入口。该组件一方面负责共享库装载，另一方面定义 Java 到 C++ 的函数映射。其核心职责包括：

- 提供 createEngine、destroyEngine 等生命周期接口。
- 提供状态注册接口，用于在 native 层创建 typed StateTable。
- 提供 ValueState、ListState、MapState、ReducingState、AggregatingState 的 typed 访问接口。
- 提供快照准备、数据导出、数据恢复和快照释放接口。

当前实现中，JNI 层不保存状态本体，但它定义了 Java 控制层访问 native 数据面的全部入口。

2.2 关键设计组件

2.2.1 StateEngine

StateEngine 是 native 数据面的最顶层组件，负责管理所有已注册状态的句柄和状态表实例。JNI 接收到一次状态访问时，先根据 state handle 找到对应 StateTable，再结合 key-group 和 namespace 路由到最终的 SwissTable。

在快照阶段，StateEngine 统一触发所有状态表进入或退出 Copy-On-Write 视图，从而保证同一次 checkpoint 周期内整套状态共享一致的快照边界。

2.2.2 状态布局与命名空间组织

状态条目在 native 侧围绕状态表、key-group、namespace 和 SwissTable 展开。其设计要点如下：

- 对于 VoidNamespace，每个 key-group 直接对应一张 $\text{SwissTable}\langle K, V \rangle$ ，访问路径最短。
- 对于一般 namespace，每个 key-group 下维护 namespace 到 $\text{SwissTable}\langle K, V \rangle$ 的映射，不同 namespace 的条目不混放在同一张表中。
- 当某个 namespace 在删除后已经为空时，系统及时移除对应空表，避免长期保留无效容器。

这种组织方式使得 checkpoint 导出、restore 重建和 namespace merge 都可以围绕统一的数据结构完成。

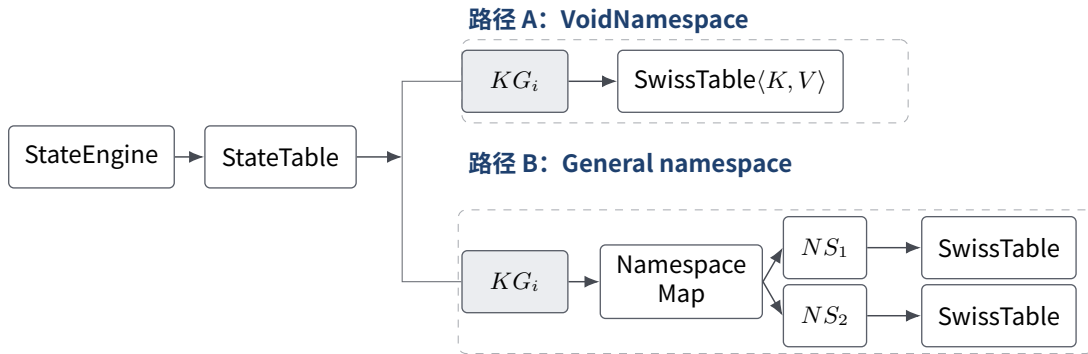


图 3 原生状态组织：VoidNamespace 路径与一般命名空间路径

2.2.3 SwissTable

$\text{SwissTable}\langle K, V \rangle$ 是状态系统的底层哈希存储结构，负责维护键到值的高效映射。其设计要点包括：

- 采用 ctrl byte 加连续 slot 布局。
- 采用 H_1 / H_2 哈希位分配方式，其中 H_2 写入 ctrl 字节， H_1 用于探测起点。
- 按 group 扫描 ctrl 区，先过滤候选槽位，再执行键比较。
- 使用 2 的幂容量、tombstone 删除和 grow/rehash 机制整理表布局。

- 这意味着 `SwissTable` 不仅是索引结构，也是实际的 `value` 的承载结构，从而减少二次寻址和额外缓存失配。

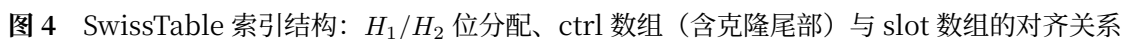


图5 SwissTable 控制区布局与分组扫描示意

2.2.4 L0 Hot-Key Cache

L0 Hot-Key Cache 是由配置显式启用的外层快路径，其核心是一块位于鲲鹏 L0 Cache 用户态分区的紧凑哈希索引。未启用或硬件门禁未通过时，权威状态仍由 SwissTable 正常服务。下面给出其数据结构与访问算法。

数据结构 缓存整体为一个 set-associative 哈希索引，共 N 个 set，每个 set 由固定的 8 个 way 组成，紧凑占用 $3 \times 64 = 192$ B，且严格按 64 B 对齐：

- **tags 行** (64 B)：存放 8 个 tag 字节以及用于轮转替换的指针 rr，其余字节填充；空 way 以 **EMPTY = 0x80** 标记，正常 tag 落在 $[0, 0x7F]$ ，与 **EMPTY** 不会冲突。
- **keys 行** (64 B)：存放 8 个 8 B 定长 key 槽位。
- **vals 行** (64 B)：存放 8 个 8 B 定长 value 槽位。

所有 64 位 key 先经 Wyhash 风格的 64 位混合函数得到一个散列值 h ，再沿用 SwissTable 的 H_1/H_2 位分配方式从 h 中切分：低 7 位 $H_2 = h \& 0x7F$ 用作 tag，高位 $H_1 = h \gg 7$ 用作 set

定位。set 索引由 $set = H_1 \& (N - 1)$ 给出 (N 为 2 的幂)。8 个 way 之间以 NEON `vceq_u8` 对 tags 行进行单周期 8-way 并行比较, 在不支持 NEON 的平台上退回标量实现。

查找算法 给定 (kg, ns, key), 首先将 key 归一化为 64 位整数, 再计算混合散列 h 以及 H_1 、 H_2 , 定位到唯一的 set:

1. 一次 64 B 载入读入 tags 行, 使用 `vceq_u8(tags, H2)` 得到 8 字节匹配掩码, 每匹配一个 way 对应一个非零字节。
2. 对掩码中每个候选 way 读取 `keys[way]`, 与 key 逐一比较; 相等则读取 `vals[way]` 返回。
3. 若无匹配或匹配 way 的 key 不等, 则记为 miss, 并回落到 SwissTable 权威存储。

在典型命中路径下, 整次访问仅触及 tags 行和命中 way 所在的 key 、 val 两个 8 B 字段, 共约 $64 + 8 + 8 = 80$ B, 全部位于 L0 内存。

插入与写穿透 写操作在 SwissTable 成功更新之后, 再同步写入 cache:

1. 按查找算法定位 set; 若存在 tag 与 key 同时命中的 way, 直接覆盖该 way 的 key/val 。
2. 否则寻找 EMPTY way 作为插入点。
3. 若 8 个 way 均被占用, 则按 set 内轮转指针 rr 选择一个 way 进行替换, 并推进 rr 。

rr 复用 tags 行尾部的 1 字节元数据空间, 仅占用低 3 位, 不增加额外 cache line。

失效算法 删除或 key -group 迁移时, 按查找算法定位 set 并匹配 way, 将命中 way 的 tag 置为 EMPTY, $keys/vals$ 不做清零。set 作为整体在 rescale 时归还 HotCacheManager 全局池, 由该管理器统一承担 L0 内存的分配与回收。

TaskManager 进程级共享管理 `libl0mempool.so` 的 `tuner` 是进程级稀缺资源, 不能按并行 `subtask` 或每个 `StateEngine` 重复初始化。当前实现因此在每个 `TaskManager` 进程内只创建一个 `HotCacheManager`, 多个 `StateEngine` 通过 `shared_ptr` 复用同一 `tuner`、同一底层 L0 分配和同一 `HotSet` 池。全局注册表使用弱引用: 最后一个 `engine` 销毁后, `manager` 与硬件资源随之释放, 不跨 `TaskManager` 进程共享。

共享后, `manager` 的分配、回收、统计与生命周期账本由互斥锁保护; 每个 `HotCacheLL` 仍由所属 Flink `subtask` 单线程访问。自适应 `rebalance` 只清理触发它的当前 state cache, 不再因为一个 state 的 miss rate 而清空其他并行 instance 的缓存。配置项 `l0-cache.expected-engines` 为兼容历史名称保留, 其计数现在应等于预期 `TaskManager` 进程数, 而不是 `slot`、`operator` 或 `StateEngine` 数。启动日志中首个 `engine` 记录 `engine_start active=1`, 同进程后续 `engine` 记录 `engine_reuse active=1`; `state_attach` 则证明至少一个受支持状态真正取得 `HotSet`。



图 6 L0 Hot-Key Cache 快路径与 HotSet 布局: JNI 入口在 L0 内完成 8-way tag 并行比对, 命中直接返回; miss 时回落 SwissTable 并写穿透回填 HotSet

2.2.5 状态访问策略与 JNI 快路径

系统通过状态访问策略和 JNI 快路径, 将高频场景映射到更短、更窄、更少复制的访问路径。设计上主要包括

- ValueState 根据 key 与 namespace 形态预计算 KeyNsStrategy。
- ListState 根据 key、namespace 和 element 类型预计算 ListStrategy。
- MapState 根据 outer key、namespace 和 inner map 形态预计算 MapStrategy。

策略一旦确定, 后续每次状态访问都只进入对应分支, 不再重复做整套类型判定。

2.2.6 透明融合边界与状态内部快路径

为避免将后端优化泄漏到用户作业或 benchmark workload 中, ForL0 State Backend 的融合优化遵循一条边界原则: 生产路径上的状态访问优化必须封装在 ForL0 后端、JNI 层和 native 状态引擎内部, 不能要求 WordCount、NexMark 或客户作业显式调用 ForL0 专有 API。因此, 系统区分两类快路径:

- **透明快路径:** 由 ForL0ValueState、ForL0MapState 等状态实现内部完成。用户仍然调用 Flink 标准的 `value()`、`update()`、`get()`、`put()`、`clear()` 等接口, 后端根据已预计算的 KeyNsStrategy / MapStrategy 选择 typed JNI、缓存或 native 特化路径。
- **诊断快路径:** 用于定位 JNI 调用粒度瓶颈的非合同实验接口, 例如 `addAndGetLong` 或顺序批量更新接口。该类接口帮助验证“多次 `get/put` 融合为一次 native 调用”能够降低 JNI 边界成本, 但不作为普通用户作业的 API 依赖。

在当前实现中，透明快路径重点覆盖 `primitive ValueState<Long>`。对于 `long/int key`、`VoidNamespace` 或 `long key + TimeWindow namespace` 的组合，`ForL0ValueState` 在内部维护一个单条目的热 `key` 缓存。缓存键由 `(key, keyGroup, namespace)` 组成；当连续访问命中同一状态条目时，`value()` 可以直接返回 Java 层缓存的 `long` 值，避免再次发起 JNI `get`。任何 `update()`、`clear()` 以及内部 `addAndGetLong()` 都会同步维护或失效该缓存，从而保证缓存只作为访问加速层，不改变 native 状态表的权威存储语义。

该设计的收益边界也较为明确：它适合热点 `key`、短期局部性强、以 `ValueState<Long>` 读改写为主的算子；对 `RowData`、`generic bytes`、复杂对象、迭代器扫描和业务函数主导的路径，系统仍保持原有 `serializer` 或 `zero-copy` 路径，不进行不透明推断。

2.2.7 v3.0 覆盖边界与离线复跑反馈

v3.0 设计进一步明确了“透明优化”和“诊断融合”的边界。当前可在普通 `workload` 中自动生效的新增能力主要是 `ForL0` 内部的 `primitive ValueState<Long>` 热 `key` 缓存；它要求状态描述符、`serializer`、`key/namespace` 形态能够落到后端已识别的 `long` 快路径。若作业实际进入 `TupleSerializer`、`RowData`、窗口复合 `accumulator`、`List/Map` 迭代、`SQL runtime` 生成状态或通用 `bytes serializer` 路径，则本轮缓存不会强行介入，以避免破坏 Flink 状态语义、`serializer` 兼容性和 `checkpoint/savepoint` 格式。

`MapState` 的 `mapAddAndGetLongLong` 与 `mapAddSequentialAndSumLongLong` 属于诊断融合 JNI：它证明了 `map-heavy workload` 在合适调用粒度下可以减少 Java/JNI/native 往返，但标准 Flink `MapState` API 只暴露 `get()`、`put()`、`entries()` 等离散操作，后端无法在不理解上层业务循环的情况下安全地把多次调用自动合并。因此，该能力不作为普通 `NexMark`、客户作业或其他生产 `workload`，需要在 `ForL0` 后端可观察的状态访问模式、Flink `runtime/operator` 生成层，或 `SQL` 代码生成层识别 `compound state operation`，而不是修改 `benchmark/workload` 源码。

2026 年 7 月的离线复跑也验证了上述边界：`WordCount` 合同场景实际使用 `sliding-window + string key + TupleSerializer` 的 `generic` 路径，不命中 `primitive ValueState<Long>` 缓存；`NexMark` 默认 `event-count` 口径中，各 `query` 大量状态由 `SQL runtime` 的窗口、`join`、`rank`、`RowData` 和复合 `accumulator` 驱动，透明缓存只能覆盖其中很小一部分访问。因此，v3.0 后续设计重点应从单点 JNI 函数继续上移到“可被后端自动识别的复合状态操作”和“SQL/窗口状态类型专用快路径”，同时继续保持 `workload` 代码零侵入。

2.2.8 类型分析与布局描述

`TypeAnalyzer` 与 native `type_layout` 共同构成 Java `serializer` 世界与 C++ 模板实例世界之间的映射层。当前重点支持的类型可分为两组：

- 基础类型：INT32、INT64、FLOAT32、FLOAT64、BOOL。
- 结构类型：STRING、BYTES、LIST、MAP、FIXED_ROW、VOID_NS 和 TIME_WINDOW。

其中，FIXED_ROW 用于固定长度的多字段 RowData key，TIME_WINDOW 用于窗口 namespace，BYTES 与 STRING 在 native 层都以连续字节容器承载，但在 checkpoint 输出时仍遵循各自 serializer 语义。

2.3 原生状态组织实现

2.3.1 结构组成

原生状态组织由四层结构组成：

- StateEngine：任务级状态引擎，维护所有状态表和状态句柄。
- StateTable<K,V>：单个状态描述符的分区状态表。
- namespace 容器：在非 VoidNamespace 模式下，将一个 key-group 下的 namespace 映射到各自的 SwissTable<K,V>。
- SwissTable<K,V>：实际保存键和值的底层哈希表。

在值组织上，slot 中直接持有 primitive、连续字节容器、列表容器、typed inner map 或 accumulator，因此结构上不再需要独立的外部 value store。

2.3.2 RowData 快路径实现

当前实现针对 RowData 提供了明确的三类路径：

- 单字段 primitive RowData key：直接解包为 long 或 int。
- 固定长度多字段 RowData key：编码为 FixedRow。
- 复杂变长 RowData key：回退到 generic 序列化路径。

对部分 RowData 或 BinaryRowData value，native 侧可以返回底层地址和长度，Java 层再构造只读内存视图，从而减少中间 byte[] 拷贝。

2.3.3 主要操作算法

围绕原生索引结构，系统的核心操作可以归纳为查询、插入、更新、删除、扩容与重哈希，以及快照一致性视

查询 (Get) 查询流程总是先在 L0 Hot-Key Cache 上尝试快路径：若 state 类型被纳入 cache 白名单且命中，则直接返回；否则定位目标状态表，再根据 key-group 和 namespace 找到对应的 SwissTable。在表内查找时，系统先扫描 ctrl 区中的 H_2 指纹以筛选候选槽位，再对少量候选槽位执行键比较；若当前探测组内已经出现 EMPTY 槽位，则说明本次查询未命中，可以直接返回空结果。SwissTable 命中后结果同步回填

cache, 从而让后续同 key 访问继续走热路径。

算法 1 状态查询算法 (含 L0 Hot-Key Cache 快路径)

输入: 状态句柄 stateId, key-group kg, namespace ns, 用户键 key

输出: 若命中则返回对应值, 否则返回 NOT_FOUND

```
1 table ← StateEngine.lookup(stateId)
2 if table.cache ≠ ∅ then                                     ▷ L0 Hot-Key Cache 快路径
3   hit, val ← table.cache.get(kg, ns, key)
4   if hit then
5     return val
6   end if
7 end if
8 if table 为 VoidNamespace 模式 then
9   swiss ← table.directTable(kg)
10 else
11   swiss ← table.namespaceTable(kg, ns)
12   if swiss = ∅ then
13     return NOT_FOUND
14   end if
15 end if
16 hash ← smear(key)
17 h1 ← hash ≫ 7, h2 ← hash & 0x7F
18 group ← probeStart(h1, swiss.capacity)
19 while true do
20   matches ← matchH2(swiss.ctrl[group], h2)
21   for all slot ∈ matches do
22     if swiss.key(slot) = key then
23       value ← swiss.value(slot)
24       if table.cache ≠ ∅ then
25         table.cache.put(kg, ns, key, value)                 ▷ miss 回填
26       end if
27       return value
28     end if
29   end for
30   if containsEmpty(swiss.ctrl[group]) then
31     return NOT_FOUND
32   end if
33   group ← nextProbe(group)
34 end while
```

插入 (Put) 插入流程首先定位目标 StateTable 和具体 namespace 对应的 SwissTable。若目标 namespace 尚未建立, 则系统先创建对应表结构。随后根据哈希结果执行槽位选择; tombstone 过多或增长预算耗尽而无法继续插入, 则先触发 rehash 或 grow, 再重试插入。SwissTable 写入成功后再向 L0 Hot-Key Cache 写穿透, 保证 SwissTable 是唯一权威来源。

更新 (Update) 更新流程与插入流程共用同一条索引定位路径。区别在于, 更新场景要求条目已经存在: 命中 slot 后, 直接覆盖旧值; 若当前 key 尚不存在, 则更新流程退化为插入流程。对于快照窗口内的更新, 系统还

删除 (Remove) 删除流程同样以目标 SwissTable 为基础执行。若目标 key 存在, 则将对对应槽位标记为 tombstone; 在一般 namespace 模式下, 如果某个 namespace 对应表在删除后已经为空, 则进一步移除该 namespace

算法 2 状态插入算法 (SwissTable 优先, L0 cache 写穿透)

输入: 状态句柄 stateId, key-group kg, namespace ns, 用户键 key, 用户值 value

输出: 将 key 对应条目写入目标状态表

```
1 table ← StateEngine.lookup(stateId)
2 swiss ← table.getOrCreateTable(kg, ns)
3 hash ← smear(key)
4 while true do
5   result ← swiss.put(hash, key)
6   if result = NEED_REHASH then
7     swiss.rehash()
8   else if result = NEED_GROW then
9     swiss.grow()
10  else
11    slot ← result & SLOT_MASK
12    swiss.value(slot) ← value
13    break
14  end if
15 end while
16 if table.snapshotActive(kg) then
17   table.recordCowMutation(kg, ns, key)
18 end if
19 if table.cache ≠ ∅ then
20   table.cache.put(kg, ns, key, value)
21 end if
```

▷ 写穿透到 L0 cache

算法 3 状态更新算法 (L0 cache 写穿透)

输入: 状态句柄 stateId, key-group kg, namespace ns, 用户键 key, 新值 value

输出: 若条目存在则原位更新, 否则按插入路径写入

```
1 table ← StateEngine.lookup(stateId)
2 swiss ← table.getOrCreateTable(kg, ns)
3 slot ← swiss.findSlot(key)
4 if slot = NOT_FOUND then
5   调用算法 2 执行插入
6   return
7 end if
8 if table.snapshotActive(kg) then
9   table.recordOldValueIfNeeded(kg, ns, key)
10 end if
11 swiss.value(slot) ← value
12 if table.cache ≠ ∅ then
13   table.cache.put(kg, ns, key, value)
14 end if
```

▷ 写穿透

表本身，从而避免长期持有空容器。与此同步，L0 Hot-Key Cache 中的对应 slot 被直接失效（invalidate），以免后续读路径读到已删除的旧值。

算法 4 状态删除、L0 cache 失效与空命名空间清理算法

输入： 状态句柄 stateId, key-group kg, namespace ns, 用户键 key

输出： 删除目标条目，同步失效 cache，并在需要时清理空 namespace 表

```

1 table ← StateEngine.lookup(stateId)
2 swiss ← table.findTable(kg, ns)
3 if swiss = ∅ then
4   if table.cache ≠ ∅ then
5     table.cache.invalidate(kg, ns, key)
6   end if
7   return
8 end if
9 if table.snapshotActive(kg) then
10  table.cowBeforeErase(kg, ns, key)
11 end if
12 swiss.erase(key)
13 if table.cache ≠ ∅ then
14  table.cache.invalidate(kg, ns, key)
15 end if
16 if table 非 VoidNamespace 模式 and swiss.isEmpty() then
17  table.removeNamespace(kg, ns)
18 end if

```

▷ 同步失效

扩容与重哈希（Grow/Rehash） 扩容与重哈希属于索引结构的内部整理过程。当前表如果只是 tombstone 累积过多，则可以在保持容量不变的情况下重新布局；如果有效条目数量已经逼近容量上限，则必须 grow。无论哪种情况，核心过程都是重新分配新表，并把旧表中的有效条目按新的布局重新插入。

算法 5 SwissTable 扩容与重哈希算法

输入： 目标哈希表 swiss

输出： 将有效条目迁移到新的布局中，并清理 tombstone

```

1 if swiss.needGrow() then
2   newCapacity ← 2 × swiss.capacity
3 else
4   newCapacity ← swiss.capacity
5 end if
6 fresh ← allocateSwissTable(newCapacity)
7 for slot ← 0 to swiss.capacity - 1 do
8   if swiss.ctrl(slot) 表示 FULL then
9     hash ← swiss.hash(slot)
10    key ← swiss.key(slot)
11    value ← swiss.value(slot)
12    fresh.insert(hash, key, value)
13   end if
14 end for
15 swiss.swap(fresh)

```

快照一致性视图遍历 快照遍历阶段并不直接输出当前表内容，而是输出“checkpoint 发起时刻”的一致性视图。对于快照开始后新增的条目，遍历时直接跳过；对于快照窗口内被覆盖或删除的条

算法 6 快照一致性视图遍历算法

输入：状态表 `table`，key-group `kg`

输出：输出 checkpoint 发起时刻的一致性条目序列

```
1 for all ns ∈ table.namespaces(kg) do
2   swiss ← table.currentTable(kg, ns)
3   for all entry ∈ swiss.currentEntries() do
4     if table.isSnapshotNew(kg, ns, entry.key) then
5       continue
6     else if table.hasOldValue(kg, ns, entry.key) then
7       emit(ns, entry.key, table.oldValue(kg, ns, entry.key))
8     else
9       emit(ns, entry.key, entry.value)
10    end if
11  end for
12  for all deleted ∈ table.deletedEntries(kg, ns) do
13    emit(ns, deleted.key, deleted.value)
14  end for
15 end for
```

2.4 ForL0State 实现

2.4.1 ForL0ValueState

ForL0ValueState 负责单值状态的读取、写回与清理。其设计重点在于根据 `key`、`namespace` 和 `value` 的组合，尽量选择最短路径完成一次状态访问。对于 `primitive` 值，会优先使用一次 JNI 调用完成存在性判断与取值；对于 `bytes` 和 `string`，会优先使用连续字节返回；对于 `RowData`，则优先判断能否走零拷贝或压缩编码路径。

在 `primitive` `long` 计数类场景中，**ForL0ValueState** 还维护一个后端内部的单条目缓存，用于覆盖 `value() -> update()` 或连续读取同一状态条目的常见访问模式。该缓存只在非 `RowData` 的 `ValueState<Long>` 快路径上启用，并记录 `key`、`key-group` 与 `namespace` 边界；命中时可跳过一次 JNI `get`。写入、清理和内部融合加法都会同步更新或失效缓存，因此 `checkpoint`、`savepoint`、`rescale` 和后续 `native` 访问仍以 `native StateTable` 为权威来源。

对于可以在后端内部安全表达为“加 `delta` 并返回新值”的场景，**ForL0ValueState** 提供 `addAndGetLong` 作为内部/诊断快路径，对应 `native` 层的 `valueAddAndGet*` 系列 JNI 入口。该入口不要求普通用户作业修改代码；普通作业仍通过 `Flink ValueState` 接口访问，后端仅在自身实现内部选择是否利用该能力。

2.4.2 ForL0ListState

ForL0ListState 负责列表状态的读取、追加和整体更新。对于 `long key` 加 `long element` 组合，系统使用专用路径减少逐元素反序列化；对于 `generic element` 类型，仍使用 `serializer` 路径保证兼容性。`namespace merge` 在语义上遵循 `Flink` 规则，只是底层值容器保持在 `native` 侧。

2.4.3 ForL0MapState

ForL0MapState 的关键设计在于 typed inner map。对常见组合，系统允许按 inner entry 粒度执行 get、put、remove 和 contains，而不是把整张 map 始终当作 opaque bytes blob 读写。只有在 generic 回退路径下，才需要通过通用序列化方式处理内层 map。

MapState<Long,Long> 是 JNI 调用粒度最敏感的路径之一。一次逻辑上的 counter 更新如果通过标准接口写成 get() 再 put(), 会产生两次 Java/JNI/native 往返。为定位该瓶颈，ForL0MapState 与 NativeEngine 增加了 mapAddAndGetLongLong 和 mapAddSequentialAndSumLongLong 诊断接口，用于将单个或一批 long inner-key 的读改写合并到 native 层完成。该设计验证了 JNI/API 粒度对 map-heavy scalar workload 的影响，但生产透明路径仍以标准 MapState 接口和 typed get/put/contains/remove 为主；若未来要对所有 workload 自动生效，需要在 Flink runtime 或算子生成层识别 compound state operation，而不是修改业务代码。

2.4.4 ForL0ReducingState 与 ForL0AggregatingState

这两类状态采用“native 存储，Java 执行用户函数”的模式。对于 ReducingState，旧值保存在 native 状态中，Java 侧执行 ReduceFunction.reduce；对于 AggregatingState，accumulator 保存在 native 状态中，Java 侧执行 AggregateFunction 的 createAccumulator、add、merge 和 getResult。这种分层保证了用户函数语义不变，同时让状态本体继续停留在 native 数据面。

3 容错机制设计

ForL0 State Backend 的容错机制遵循 Flink Checkpoint / Savepoint 的一致性语义。系统的 KV 状态快照来源于 native StateTable 的一致性视图，Priority Queue 状态仍沿用 Flink 原生写出机制，最终共同组成完整的 Keyed State 快照结果。

3.1 快照发起

在 Flink 发起 snapshot 后，系统首先进入同步准备阶段。Java 层通过 ForL0SnapshotStrategy 调用 NativeEngine.prepareSnapshot，native StateEngine 随后将所有相关 StateTable 切换到 Copy-On-Write 模式。与此同时，Java 层构造 ForL0SnapshotResources，收集 KV state meta info、priority queue snapshot 资源以及异步写出所需的上下文。

这一阶段的目的是不是执行序列化，而是建立一份稳定的“快照边界”，保证异步写出线程读取的是 checkpoint 发起时刻的一致性视图。

3.2 快照采集

快照采集阶段按 key-group 写出数据，主要流程如下：

1. 写出 `KeyedBackendSerializationProxy`，记录 key serializer snapshot 与 state meta info snapshot。
2. 为每个 key-group 记录输出偏移并写出 priority queue 状态块。
3. 调用 native checkpoint 写出逻辑，导出该 key-group 下全部 KV state 数据。
4. 所有 key-group 完成后，调用 `NativeEngine.releaseSnapshot` 释放快照上下文。

native KV block 的内部组织以 state 为单位记录 stateId、entryCount 和具体条目内容。namespace、key 和 value 的输出格式与 Flink serializer 语义保持兼容，从而保证 restore 端可以按统一格式重建状态。

3.3 Copy-On-Write 一致性模型

当前实现的 Copy-On-Write 不是复制整张状态表，而是记录“自快照开始之后发生变化的条目”。其基本思想

- 快照开始时，在每个 key-group 上打开快照活动标记。
- 若某个条目在快照窗口内第一次被修改，则先保存旧值，再执行更新。
- 若某个条目是在快照开始后新增，则记录为“快照后新增”，本轮快照中不输出。
- 若某个条目在快照窗口内被删除，则记录其快照时刻旧值，以便异步遍历时仍能写出。

异步遍历阶段并不是简单遍历当前表，而是在当前表内容之上叠加覆盖记录、删除记录和新增标记，从而 namespace 状态，上述过程按 namespace 分别进行。



图 7 Copy-On-Write 快照一致性模型：前台表 $\oplus$ 按 key-group 的 overlay 还原 t_0 视图

3.4 Savepoint 与状态恢复

ForL0 State Backend 同时支持普通 checkpoint 恢复与 canonical savepoint 恢复。恢复流程由 `ForL0KeyedStateBackendBuilder` 统一驱动，主要包括以下步骤：

1. 读取状态句柄并恢复 `KeyedBackendSerializationProxy`。

2. 执行 key serializer 兼容性校验, 重建 state meta info registry。
3. 恢复 priority queue 状态。
4. 按 key-group 读取 KV 数据块, 并写回对应 typed `StateTable`。
5. 对 canonical savepoint, 将规范格式条目翻译为 native 状态表中的键值结构。

恢复完成后, backend 重新持有全部 engine handle 与 state handle, 后续访问继续通过 Java 控制层和 JNI 路由到 native 数据面。

3.5 资源回收与异常处理

为避免 native 资源泄漏, 当前实现通过显式生命周期控制保障异常路径下的资源回收:

- snapshot 正常完成后释放 Copy-On-Write 快照状态。
- snapshot 失败时, 异常路径仍尝试释放快照上下文。
- backend 构建或 restore 失败时, builder 负责销毁已创建的 native engine。
- backend dispose 时调用 `NativeEngine.destroyEngine` 释放状态表、容器和内部缓存资源。

4 插件式集成设计

ForL0 State Backend 以插件方式接入 Flink。系统通过 `StateBackendFactory` 完成后端工厂注册, 通过 JNI 动态加载 native 状态引擎, 并通过 Flink 标准 backend 构建流程完成运行时接入。

4.1 加载机制

ForL0 的加载机制分为两部分:

1. Flink 通过 `ForL0StateBackendFactory` 创建 `ForL0StateBackend` 实例。
2. `ForL0KeyedStateBackendBuilder` 在构建阶段调用 `NativeEngine.ensureLoaded()` 装载 native 共享库。

native 共享库的加载采用两级策略: 优先从系统库路径加载; 若系统库路径不可用, 则从打包资源中提取。

4.2 运行时接入边界

在运行时接入方面, ForL0 State Backend 保持了以下边界约束:

- 对 Flink 上层用户代码透明, 用户状态访问方式不因后端替换而改变。
- 对 Flink 内核源码无侵入, 不修改 Flink 原生状态接口语义。
- Priority Queue 保持原有行为, Keyed State 则切换到 ForL0 自身的 native 数据面。

因此，系统的集成重点在于状态后端实现与运行时生命周期协调，而不是对用户 API 做扩展。

5 总结

ForL0 State Backend 采用三层架构：Java 控制层负责状态接口适配、状态注册以及与 Flink 运行时的生命周期对接；JNI 桥接层负责原生库加载、句柄管理和按状态类型分发的访问入口；C++ 原生状态引擎承担权威状态存储、命名空间组织、SwissTable 哈希结构以及 Copy-On-Write 快照。整个状态主数据面完全收敛在 native 侧。

在这一分层之上，系统围绕热点访问路径组织了若干结构：typed StateTable 与 SwissTable 提供 SWAR 并行探测的哈希索引，RowData 快路径、typed inner map、TimeWindow 专用路径针对常见键与值形态减少复制与装箱，L0 Hot-Key Cache 作为 8-way set-associative 读写穿透缓存承接高频访问，CoW 快照与 Flink 的 Checkpoint、Savepoint 语义保持一致。

ForL0 State Backend 与 Flink 状态接口和容错机制保持兼容，用户侧无需修改即可使用；同时在哈希组

v3.0 进一步明确透明优化的覆盖边界：当前可自动作用于普通作业的新增优化以 primitive ValueState<Long> 热 key 缓存为主，MapState 批量加法等融合 JNI 仍定位为诊断能力。离线复跑表明，WordCount sliding-window、NexMark SQL 窗口/join/rank 以及客户双流 join 中的大量状态访问会进入 TupleSerializer、RowData、复合 accumulator、迭代器或业务对象主导路径，不能仅依赖单点 JNI 函数获得全 workload 收益。后续若继续保持 workload 零侵入，设计重点应转向 ForL0 后端和 Flink runtime/operator/SQL codegen 层可自动识别的 compound state operation，以及窗口、join、rank、MapState/ListState 迭代等复杂状态形态的专用透明快路径。

实验编排层也在 v3.0 中补充运行保障设计：离线一键脚本在进入实验前校验 TaskManager 与 slot 数，NexMark driver 运行期间继续监控 Flink REST 中的 FAILED job、TaskManager 数和 slot 数；一旦集群退化即终止 driver 并失败退出，避免在离线机器上因无限 stop/等待消耗实验窗口。--full 模式的 NexMark pressure suite 固定展开为 forl0_no_full_gc_allq_pressure、forl0_no_full_gc_q8_q11_deep 和 forl0_no_full_gc_lateq_deep；同时提供 --pressure-only，用于离线机器只补跑 ForL0/L0 友好的 pressure 配置。该逻辑只影响 benchmark 编排，不改变 ForL0 状态后端语义或 workload 源码。

离线服务器复现交付报告中的 Ascend 编号化性能清单时，推荐使用仓库根目录下的单一入口：

```
cd /root/forL0
./reproduce-all
```

该入口后台启动独立 `campaign`，先运行 `HashMap/ForL0` 隔离的 `smoke` 正确性门禁，通过后再执行 `W01-W02、N01-N14、C01-C08` 正式矩阵；运行日志和中间证据写入 `benchmark/results/runs/<run_id>/.logs` 及其 `smoke/、formal/` 子目录。只有整个 `campaign` 成功才原子发布到无子目录的 `benchmark/results/latest/`，并通过 `UPLOAD_MANIFEST.tsv` 保存扁平文件名到原始相对路径的映射；失败 `campaign` 保留 `FAILED.txt` 和局部证据，但不会覆盖 `latest`。

当前正式 `NexMark` 矩阵是面向交付时间与稳定性的代表性集合，成对运行 `q3/q4/q9/q18/q19/q20`，并非 `q0-q22` 的穷举测试。仓库包含 `q0-q22` 的 `HashMap/ForL0 SQL` 资产和可选 `scenario`；若验收要求全 `query` 覆盖，应单独执行完整扫描，并将其结果与默认代表性性能矩阵分开陈述。

2026-07-11 的 `Ascend` 离线复现进一步补充了工程化约束：`ForL0` 的 `initial-table-capacity`、`L0 cache size` 等后端配置必须从 `Flink Java` 配置完整上传到 `native StateEngine`，否则 `benchmark.yaml` 中的调优参数只是表面生效；`native` 共享库的构建必须将头文件纳入依赖，避免 `JNI/native` 结构体布局变化后只重编部分 `object` 引入崩溃；离线部署必须保证 `Flink lib/` 中的 `backend jar、jar` 内嵌 `native resource、native/` 目录和 `deploy` 目录四者版本一致。上述改造均限定在 `ForL0` 后端、`native` 构建与一键实验编排范围内，不要求 `WordCount、NexMark` 或客户用例修改源码。

同一轮复现还暴露了采集开关必须真正进入后端的问题：早期脚本无条件传入 `forL0.metricsCollector.enabled=true`，但后端 `builder` 并未读取该配置，`HotCache gauge` 会在有 `MetricGroup` 时无条件注册。`v3.0` 将该开关纳入 `ForL0` 配置面，默认性能复现关闭，仅在诊断或指标采集时显式打开。该设计避免性能交付口径 `JNI gauge` 调用影响，同时保留运维观测能力。

由于 `Ascend` 容器环境下短时 `WordCount` 单次运行存在明显方差，`v3.0` 设计将“实验复现口径”也视为交付设计的一部分：编号化一键脚本允许场景声明 `repeat_runs` 与 `repeat_policy`，当前 `WordCount fastpath` 场景采用同并行度 `p4` 的 `best-of-3`，并在 `raw JSON` 中保留每次 `job id、吞吐和耗时`。这一机制不改变后端实现，也不只对 `ForL0` 生效；`HashMap` 与 `ForL0` 使用同一 `repeat` 规则，从而降低离线机器单次坏样本导致结论翻转的风险。

`Ascend` 默认编号清单采用总吞吐优先原则，但该原则不改变公平后端对比边界：同一 `workload` 内 `HashMap` 与 `ForL0` 必须使用相同 `query、输入比例、Flink/operator` 并行度、`slot` 数和监控窗口；`ForL0` 只允许通过自身后端参数和实现优化取得收益。2026-07-12 最终一键复跑在 `benchmark/results/run_logs/ascend_reproduction_20260712_040246.tsv` 中完整通过，默认清单覆盖 `WordCount、NexMark q18/q19/q20/q9/q4/q3` 以及 `Client usecase`。该轮 `WordCount p4 best-of-3` 为 `831,805 vs 1,039,194 records/s/core (+24.9%)`；`NexMark` 中 `q18 tps_probe` 为 `174.90 K/s vs 215.89 K/s (+23.4%)`，`q18 lateq_deep` 为 `947.15 K/s vs 1.22 M/s (+28.8%)`，`q19`

为 +3.6%, q20 为 +3.5%, q9 为 +8.8%, q4 在 600 K/s、45 s 稳定窗口下为 +11.9%, q3 为 +16.8%。其中 q20/q9/q19 同时体现更显著的 CPU/core 效率改善。Client 原始业务形态主要作为兼容与无回退复核, `scalar_state_probe_2m_ops64_batch` 则作为 map-heavy 诊断压力场景, 24,869 vs 62,049 records/s/core (+149.5%)。q5、q8、q11、state 和 `benchset` 不进入默认清单: 前者未形成稳定、显著、公平的 ForL0 吞吐优势, 后者会拖慢或削弱离线一键复现的确定性。曾探索过的 ForL0 `higher-parallelism` q19/q20 配置虽然面向总 TPS, 但改变了 operator 级执行拓扑, 已从默认编号清单、YAML 场景和交付结论中撤出。

2026-08 的硬件复跑诊断进一步修正了两个实现与口径问题。第一, L0 tuner 实际按 TaskManager 进程生效, 旧实现按每个 StateEngine 创建 manager 会在多 operator/多 subtask 下耗尽 tuner; 现已改为进程内共享 manager, 并将并发账本与 per-state rebalance 隔离。第二, WordCount `stateful_counter_p4_probe` 使用融合 `addAndGetLong`: 它在一次 JNI 内直接 `modify_in_place` 权威 SwissTable, 只有写穿透而没有 HotCache get, 因此观测到 `lookups=0,hits=0` 表示“未执行缓存查询”, 不是 0% 命中。该场景现明确关闭 L0, 避免为无读取收益的 cache 分配硬件容量; 它用于衡量融合 JNI、primitive SwissTable 与低 GC 路径, 而 L0 硬件收益由具有非零 lookups/hits 的 Client/NexMark L0-on 场景证明。